

Assignment 2

Diabetes Prediction using Machine Learning

Part 1 — Data Preparation

1.1 Dataset Exploration Summary

The dataset was loaded using the Pandas library and explored to understand its structure and characteristics.

Dataset Shape

The dataset contains:

- **768 rows**
- **9 columns**

Therefore, there are 768 patient records available for the analysis.

Column Names

The dataset contains the following columns:

1. Pregnancies
2. Glucose
3. BloodPressure
4. SkinThickness
5. Insulin
6. BMI
7. DiabetesPedigreeFunction
8. Age
9. Outcome

Data Types

Most variables in the dataset are numerical.

- Integer (`int64`) variables include Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, Age, and Outcome.
- Floating-point (`float64`) variables include BMI and DiabetesPedigreeFunction.

Basic Statistical Summary

A statistical summary was generated using the `describe()` function. Some important observations include:

- The average Glucose value is approximately **120.89**.
- The average BMI is approximately **31.99**.
- The average age of patients is approximately **33.24 years**.
- The Outcome mean is approximately **0.349**, indicating that around 35% of the records belong to the diabetes-positive class.

1.2 Features and Target Variable

The dataset was divided into input features and the target variable.

Features (X)

The following eight columns were used as input features:

- Pregnancies
- Glucose
- BloodPressure
- SkinThickness
- Insulin
- BMI
- DiabetesPedigreeFunction
- Age

Target Variable (y)

The **Outcome** column was selected as the target variable.

- `0` = No Diabetes
- `1` = Diabetes

1.3 Preprocessing Steps Performed

The following preprocessing activities were considered and performed:

- The dataset was checked for missing values.

- No formally missing values were found.
- All features were numerical, so categorical encoding was not required.
- The data was separated into features and target variables.
- The dataset was divided into training and testing datasets.

1.4 Training and Testing Dataset Sizes

After performing the train-test split:

- **Training dataset:** 652 records with 8 features
 - **Testing dataset:** 116 records with 8 features
-

Part 2 — Machine Learning Model Building

2.1 Models Implemented

Three Machine Learning classification algorithms were implemented:

1. **Logistic Regression**
2. **Decision Tree Classifier**
3. **Random Forest Classifier**

All models were trained using the prepared training dataset.

2.2 Logistic Regression

Logistic Regression is suitable for binary classification problems because the target variable has two possible classes: diabetes and no diabetes.

The model estimates the probability that a patient belongs to the diabetes-positive class based on the input medical features.

2.3 Decision Tree Classifier

The Decision Tree Classifier makes predictions by splitting the data into branches based on feature values. It learns decision rules from the training data and classifies patients into either the diabetes or non-diabetes category.

A Decision Tree can capture non-linear relationships between the input features and the target variable.

2.4 Random Forest Classifier

The Random Forest Classifier is an ensemble learning algorithm that combines predictions from multiple decision trees.

Each tree makes a prediction, and the final prediction is determined using the combined results of the trees. Random Forest is suitable for this problem because it can model complex relationships and generally provides more robust predictions than a single decision tree.

2.5 Model Performance Comparison

The performance of the three models on the testing dataset was compared.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.818966	0.738095	0.756098	0.746988
Decision Tree	0.801724	0.687500	0.804878	0.741753
Random Forest	0.827586	0.733333	0.804878	0.767442

Based on the comparison, **Random Forest was selected as the best-performing model** because it achieved the highest accuracy and F1-score among the implemented models.

Part 3 — Model Training

3.1 Model Training

The selected Random Forest model was trained using the prepared training dataset.

The model was successfully fitted using the training features and target values.

3.2 Training Predictions

Predictions were generated using the training dataset to evaluate how well the Random Forest model learned the patterns present in the training data.

3.3 Training Performance

The training performance of the Random Forest model was as follows:

Metric	Training Performance
Accuracy	0.967791
Precision	0.985849
Recall	0.920705
F1-Score	0.952164

The training accuracy is approximately **96.78%**, which indicates that the model performs very well on the training data.

3.4 Overfitting and Underfitting Analysis

The training performance was compared with the testing performance.

Metric	Training	Testing
Accuracy	0.967791	0.827586
Precision	0.985849	0.733333
Recall	0.920705	0.804878
F1-Score	0.952164	0.767442

The model performs significantly better on the training dataset than on the testing dataset. Therefore, the results indicate **potential overfitting**.

This means that the model has learned the training data very effectively but its performance decreases when predicting previously unseen test data.

Part 4 — Model Evaluation

4.1 Test Dataset Performance

The selected Random Forest model was evaluated using the testing dataset.

The following results were obtained:

Metric	Test Performance
Accuracy	0.827586
Precision	0.733333
Recall	0.804878
F1-Score	0.767442

The model achieved a test accuracy of approximately **82.76%**.

4.2 Interpretation of Performance Metrics

Accuracy

Accuracy represents the overall proportion of correct predictions.

The model achieved an accuracy of approximately **82.76%**, showing reasonably good overall predictive performance.

Precision

The precision value is approximately **73.33%**.

This indicates how many patients predicted as diabetes-positive were actually diabetes-positive.

Recall

The recall value is approximately **80.49%**.

Recall is particularly important in a diabetes prediction problem because it measures the model's ability to correctly identify patients who actually have diabetes.

F1-Score

The F1-score is approximately **76.74%**.

This metric provides a balance between Precision and Recall.

4.3 Confusion Matrix

The confusion matrix obtained from the test dataset is:

Actual / Predicted	No Diabetes	Diabetes
No Diabetes	63	12
Diabetes	8	33

Therefore:

- True Negatives (TN): 63
- False Positives (FP): 12
- False Negatives (FN): 8
- True Positives (TP): 33

4.4 Confusion Matrix Interpretation

The model correctly identified:

- 63 patients without diabetes.
- 33 patients with diabetes.

However:

- 12 patients without diabetes were incorrectly predicted as having diabetes.
- 8 patients with diabetes were incorrectly predicted as not having diabetes.

The False Negative value is particularly important because a patient with diabetes may be incorrectly classified as not having diabetes.

Part 5 — Model Performance Analysis

5.1 Most Important Performance Metric

For a diabetes prediction problem, **Recall** is considered one of the most important performance metrics.

Recall measures the ability of the model to correctly identify patients who actually have diabetes.

A high Recall is important because failing to identify a patient with diabetes can be more serious than incorrectly identifying a patient as potentially having diabetes. Such patients may require further medical testing, whereas a missed diabetes case could delay further diagnosis.

The Random Forest model achieved a Recall of approximately **80.49%**, which means that it successfully identified a large proportion of the actual diabetes-positive cases.

5.2 Overall Model Performance

The Random Forest model performed reasonably well on the testing dataset.

Key test results are:

- Accuracy: **82.76%**
- Precision: **73.33%**
- Recall: **80.49%**
- F1-Score: **76.74%**

Compared with Logistic Regression and Decision Tree, Random Forest achieved the best overall combination of Accuracy and F1-Score.

However, the difference between training accuracy (**96.78%**) and testing accuracy (**82.76%**) suggests potential overfitting.

5.3 Key Findings

The main findings of this project are:

1. The dataset contains 768 records and 8 input features with Outcome as the target variable.
2. No formally missing values were found in the dataset.
3. Three Machine Learning models were implemented and compared.
4. Random Forest achieved the best overall performance among the three models.
5. The Random Forest model achieved approximately 82.76% test accuracy.
6. The model achieved a Recall of approximately 80.49%, indicating good ability to identify diabetes-positive patients.
7. The training performance was significantly higher than the testing performance, suggesting potential overfitting.

5.4 Model Limitations

The following limitations were identified:

1. Potential Overfitting

The training accuracy is approximately 96.78%, while the testing accuracy is approximately 82.76%. This difference suggests that the model may have learned the training data too closely.

2. Limited Dataset Size

The dataset contains only 768 records. A larger and more diverse dataset may improve the ability of the model to generalize to new patients.

3. False Negative Predictions

The model produced 8 False Negative predictions. In a diabetes prediction application, incorrectly predicting that a patient does not have diabetes could be an important limitation.

4. Scope of Available Features

The prediction is based only on the available medical features in the dataset. Additional relevant patient information may improve prediction performance.

5.5 Recommendations for Improvement

The following approaches can be used to improve the model:

1. Hyperparameter Tuning

The Random Forest model parameters can be optimized using techniques such as GridSearchCV or RandomizedSearchCV. Parameters such as the number of trees, maximum tree depth, and minimum samples per split can be tuned.

2. Feature Engineering and Feature Selection

New meaningful features can be created, and less useful features can be removed. This may reduce noise and improve model generalization.

3. Cross-Validation

Cross-validation can be used to evaluate the model on multiple splits of the dataset and obtain a more reliable estimate of model performance.

4. Handling Class Imbalance

The class distribution can be further analyzed. Techniques such as class weighting or resampling can be considered if class imbalance affects the model's ability to identify diabetes-positive patients.

5. Try Other Classification Algorithms

Other algorithms, such as Support Vector Machines, Gradient Boosting, or XGBoost, could be evaluated and compared with the current models.

Screenshot of Outputs

Assignment 2: Diabetes Prediction using Machine Learning

Part 1 — Data Preparation

1. Load the dataset using Pandas.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
data = pd.read_csv('/content/diabetes.csv')
data.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

2. Explore the dataset:

a. Shape of the dataset

```
data.shape
```

```
(768, 9)
```

b. Column names

```
data.columns
```

```
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',  
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],  
      dtype='object')
```

c. Data types

```
data.dtypes
```

	0
Pregnancies	int64
Glucose	int64
BloodPressure	int64
SkinThickness	int64
Insulin	int64
BMI	float64
DiabetesPedigreeFunction	float64
Age	int64

Activate Windows
Go to Settings to activate Windows.

d. Missing values, if any

```
data.isnull().sum()
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

```
dtype: int64
```

e. Basic statistical summary

```
data.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.684160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

3. Separate the dataset into:

a. Features (X)

b. Target Variable (y)

```
X = data.drop(['Outcome'], axis =1) #input columns excluding outcome
y = data['Outcome']
```

```
X.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	6	148	72	35	0	33.6	0.627	50
1	1	85	66	29	0	26.6	0.351	31
2	8	183	64	0	0	23.3	0.672	32
3	1	89	66	23	94	28.1	0.167	21
4	0	137	40	35	168	43.1	2.288	33

Next steps: [Generate code with X](#) [New interactive sheet](#)

```
y.head()
```

	Outcome
0	1
1	0
2	1
3	0
4	1

dtype: int64

4. Perform a Train-Test Split

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.15)
```

```
X_train.shape
```

(652, 8)

```
X_test.shape
```

(116, 8)

Part 2 — Machine Learning Model

Logistic Regression

```
# Import libraries
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
logistic_model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(
        max_iter = 1000,
        random_state=42
    ))
])
```

Decision Tree Classifier

```
from sklearn.tree import DecisionTreeClassifier

decision_tree_model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("classifier", DecisionTreeClassifier(
        max_depth = 5,
        random_state=42
    ))
])
```

Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier

random_forest_model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("classifier", RandomForestClassifier(
        n_estimators = 200,
        max_depth=8,
        random_state=42
    ))
])
```

Train all the models

```
models = {
    "Logestic Regression": logistic_model,
    "Decision Tree": decision_tree_model,
    "Random Forest": random_forest_model
}

for name, model in models.items():
    model.fit(X_train, y_train)
    print(f"{name} trained successfully.")

Logestic Regression trained successfully.
Decision Tree trained successfully.
Random Forest trained successfully.
```

Generate Predictions

```
predictions = {}
for name, model in models.items():
    predictions[name] = model.predict(X_test)
```

Evaluate Different Models

```
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score )

results =[]
for name, model in models.items():
    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test,y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test,y_pred)

    results.append([
        "Model":name,
        "Accuracy":accuracy,
        "Precision":precision,
        "Recall":recall,
        "F1-Score":f1
    ])

results_df = pd.DataFrame(results)
print(results_df)
```

	Model	Accuracy	Precision	Recall	F1-Score
0	Logestic Regression	0.818966	0.738095	0.756098	0.746988
1	Decision Tree	0.801724	0.687500	0.804878	0.741573
2	Random Forest	0.827586	0.733333	0.804878	0.767442

From this Random Forest is the Best model

Part 3 — Model Training

Fit the selected model using the training dataset.

```
random_forest_model.fit(X_train, y_train)
print(f"Random forest model trained successfully.")

Random forest model trained successfully.
```

Generate predictions on the training dataset

```
y_train_pred = random_forest_model.predict(X_train)
print("First 10 Training Predictions")
print(y_train[:10])
```

```
First 10 Training Predictions
340    0
689    1
312    1
151    0
400    1
268    0
220    1
556    0
330    0
597    0
Name: Outcome, dtype: int64
```

Evaluate the model's training performance.

```
accuracy = accuracy_score(y_train,y_train_pred)
precision = precision_score(y_train, y_train_pred)
recall = recall_score(y_train, y_train_pred)
f1 = f1_score(y_train,y_train_pred)

print("Training data Performance")
print(f"Accuracy : {accuracy}")
print(f"Precision : {precision}")
print(f"Recall : {recall}")
print(f"F1-Score : {f1}")
```

```
Training data Performance
Accuracy : 0.9677914110489447
Precision : 0.9858490568037735
Recall : 0.920704845814978
F1-Score : 0.9521640091116174
```

Generate predictions on the Test dataset and Evaluating the Performance

```
y_test_pred = random_forest_model.predict(X_test)
accuracy_test = accuracy_score(y_test,y_test_pred)
precision_test = precision_score(y_test, y_test_pred)
recall_test = recall_score(y_test, y_test_pred)
f1_test = f1_score(y_test,y_test_pred)

print("Test data Performance")
print(f"Accuracy : {accuracy_test}")
print(f"Precision : {precision_test}")
print(f"Recall : {recall_test}")
print(f"F1-Score : {f1_test}")
```

```
Test data Performance
Accuracy : 0.8275862068965517
Precision : 0.7933333333333333
Recall : 0.8048780487804879
F1-Score : 0.7674418604651163
```

Activate Windows

Check for potential overfitting or underfitting by comparing training and testing performance.

Comparison Table

```
performance_comparison = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1-Score"
    ],
    "Training": [
        accuracy,
        precision,
        recall,
        f1
    ],
    "Testing": [
        accuracy_test,
        precision_test,
        recall_test,
        f1_test
    ]
})

performance_comparison
```

	Metric	Training	Testing
0	Accuracy	0.967791	0.827586
1	Precision	0.985849	0.733333
2	Recall	0.920705	0.804878
3	F1-Score	0.952164	0.767442

Activate Windows
Go to Settings to activate Windows.

```
accuracy_difference = accuracy - accuracy_test
f1_difference = f1 - f1_test
if accuracy > 0.90 and accuracy_difference > 0.10 :
    print("Potential overfitting")
elif accuracy < 0.70 and accuracy_test < 0.70:
    print("Potential underfitting")
else:
    print("No potential Overfitting or Underfitting")

Potential overfitting
```

Part 4 — Model Evaluation

1. Generate predictions on the test dataset.
2. Evaluate the model using:
 - Accuracy Score
 - Precision
 - Recall
 - F1-Score
 - Confusion Matrix
3. Analyze the confusion matrix and identify:
 - True Positives
 - True Negatives
 - False Positives
 - False Negatives
4. Compare the training and testing performance of the model.

4. Compare the training and testing performance of the model.

```
from sklearn.metrics import confusion_matrix
print(f"Accuracy : {accuracy_test}")
print(f"Precision : {precision_test}")
print(f"Recall : {recall_test}")
print(f"F1-Score : {f1_test}")
cm = confusion_matrix(y_test, y_test_pred)
print(f"Confusion Matrix")
print(f"-----")
print(cm)

Accuracy : 0.8275862068965517
Precision : 0.7333333333333333
Recall : 0.8048780487804879
F1-Score : 0.7674418604651163
Confusion Matrix
-----
[[63 12]
 [ 8 33]]
```